

MODULE 3

Décider qui peut quoi

Module 3. Décider qui peut quoi

Durée : 1h30 | **Application :** TaskFlow (Node.js + Express + SQLite, port 3000) **Objectif :** Corriger toutes les failles d'autorisation, IDOR, JWT mal signé, middleware admin défaillant.

Failles présentes dans l'app (à corriger dans ce module)

#	FAILLE	FICHIER CONCERNÉ
1	JWT signé avec secret "secret123" en dur	auth.js
2	Pas d'expiration du JWT	auth.js
3	Route GET /tasks/:id , pas de vérification task.user_id === req.user.id (IDOR)	routes/tasks.js
4	Route /admin/users , middleware vérifie if (req.user) au lieu du rôle	routes/admin.js
5	JWT stocké en localStorage (XSS-vulnérable) côté client	public/app.js

1. Sessions vs Tokens (JWT)

Concept Une **session** est stockée côté serveur : le serveur garde l'état de chaque utilisateur connecté (stateful). Un **token JWT** est auto-contenu : toutes les informations sont dans le token, le serveur ne stocke rien (stateless). JWT favorise la scalabilité mais transfère la responsabilité de validation côté serveur à chaque requête.

Risque concret Si l'app mélange les deux approches ou que la stratégie n'est pas claire, les vérifications d'identité peuvent être incohérentes ou contournées.

Exemple de faille

```
// ⚠️ Faille : app utilise JWT mais vérifie aussi req.session, comportement
imprévisible
if (req.session.user || req.user) {
  // accès accordé sans savoir lequel est valide
}
```

Action pratique Ouvrir `auth.js` et `middleware/authenticate.js`. Confirmer que l'app utilise exclusivement JWT (aucune référence à `express-session`). Documenter : “cette app est stateless JWT-only”.

Test de validation

```
# Vérifier qu'aucun cookie de session n'est créé
curl -v -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' 2>&1 | grep -i "set-cookie"
# Résultat attendu après correction : Set-Cookie contient httpOnly, pas de session
id
```

Trace rapport > “L'application utilise une stratégie JWT stateless. Aucune session serveur n'est maintenue. La stratégie d'authentification est uniforme sur l'ensemble des routes.”

2. Structure d'un JWT

Concept Un JWT est composé de trois parties encodées en Base64URL, séparées par des points : `header.payload.signature`. Le **header** indique l'algorithme (`HS256`, `RS256`). Le **payload** contient les claims (données : `userId`, `role`, `exp`). La **signature** garantit l'intégrité, elle est calculée avec le secret. Le contenu du payload est **lisible** par quiconque : ne jamais y mettre de données sensibles (mot de passe, CC).

Risque concret Un attaquant peut décoder le payload (Base64 ≠ chiffrement) et observer les données. Si l'algorithme est `"alg": "none"`, certaines bibliothèques acceptent un token sans signature.

Exemple de faille

```
// ⚠️ Faille : algorithme "none" accepté par certaines configs
const token = jwt.sign({ userId: 1, role: 'admin' }, '', { algorithm: 'none' });
// Token forgé sans secret, accepté si la vérification n'impose pas l'algorithme
```

Action pratique 1. Copier un token existant depuis Postman ou la réponse `/login`. 2. Aller sur jwt.io, coller le token. 3. Observer le payload : noter les champs présents (`userId`, `role`, présence ou absence de `exp`). 4. Vérifier dans `auth.js` que `jwt.verify()` impose bien `algorithms: ['HS256']`.

Test de validation

```
# Générer un token "none" et tenter l'accès
HEADER=$(echo -n '{"alg":"none","typ":"JWT"}' | base64 | tr -d '=' | tr '+/' '-_')
PAYLOAD=$(echo -n '{"userId":1,"role":"admin"}' | base64 | tr -d '=' | tr '+/' '-_')
TOKEN_NONE="${HEADER}.${PAYLOAD}."
curl http://localhost:3000/admin/users \
  -H "Authorization: Bearer ${TOKEN_NONE}"
# Résultat attendu après correction : 401 Unauthorized
```

Trace rapport > “La structure JWT a été auditée sur jwt.io. Le payload expose userId et role (non sensible). La vérification impose l’algorithme HS256, les tokens ‘alg:none’ sont rejetés.”

3. Secret de signature

Concept Le secret JWT est la clé de confiance : quiconque le connaît peut forger n’importe quel token valide. Un secret court ou prévisible ("secret" , "123456" , nom du projet) peut être bruteforcé en quelques secondes avec des outils comme `hashcat` ou des listes de secrets JWT connus. Le secret doit être long (≥ 32 octets), aléatoire, et stocké exclusivement en variable d’environnement.

Risque concret Avec "secret123" , un attaquant peut forger un token `{ role: "admin" }` en quelques secondes et accéder à toutes les routes protégées.

Exemple de faille

```
// ⚠️ Faille : secret en dur, trop court, prévisible
const token = jwt.sign({ userId: user.id, role: user.role }, 'secret123');
```

Action pratique

Étape 1, Générer un secret fort :

```
openssl rand -base64 32
# Exemple de sortie : K7gNU3sdo+OL0wNhqoVWhr3g6s1xYv72oL/pe/Unols=
```

Étape 2, Créer ou compléter `.env` à la racine de `vuln-app/` :

```
# vuln-app/.env
JWT_SECRET=K7gNU3sdo+OL0wNhqoVWhr3g6s1xYv72oL/pe/Unols=
```

Étape 3, Modifier `auth.js` :

```
// AVANT (faille)
const token = jwt.sign({ userId: user.id, role: user.role }, 'secret123');

// APRÈS (corrigé)
const token = jwt.sign(
  { userId: user.id, role: user.role },
  process.env.JWT_SECRET,
  { algorithm: 'HS256', expiresIn: '1h' }
);
```

Étape 4, Vérifier que `.env` est dans `.gitignore` :

```
grep ".env" vuln-app/.gitignore || echo ".env" >> vuln-app/.gitignore
```

Test de validation

```
# Forger un token avec "secret123", doit être rejeté
FORGED=$(node -e "const jwt=require('jsonwebtoken');
console.log(jwt.sign({userId:1,role:'admin'},'secret123'))")
curl http://localhost:3000/admin/users \
  -H "Authorization: Bearer ${FORGED}"
# Résultat attendu après correction : 401 Unauthorized (signature invalide)
```

Trace rapport > “Le secret JWT ‘secret123’ a été remplacé par un secret de 32 octets généré via openssl. Le secret est stocké dans la variable d’environnement JWT_SECRET et exclu du dépôt git via .gitignore.”

4. Expiration du token

Concept Un JWT sans expiration (`exp`) est valide indéfiniment. Si un token est compromis (XSS, fuite de logs, MITM), l’attaquant dispose d’un accès permanent sans que le serveur puisse le révoquer (JWT est stateless). L’expiration courte (`1h` pour les accès courants, `15m` pour les routes sensibles) limite la fenêtre d’exploitation. Un refresh token avec rotation peut prolonger la session sans exposer le token principal.

Risque concret Un token volé aujourd’hui donne accès dans 6 mois si aucune expiration n’est définie.

Exemple de faille

```
// ⚠️ Faille : pas d'expiration, token éternel
const token = jwt.sign(
  { userId: user.id, role: user.role },
  process.env.JWT_SECRET
);
// Le payload ne contiendra jamais de champ "exp"
```

Action pratique

```
// APRÈS (corrigé), dans auth.js ou routes/auth.js
const token = jwt.sign(
  { userId: user.id, role: user.role },
  process.env.JWT_SECRET,
  { algorithm: 'HS256', expiresIn: '1h' } // ← ajout
);
```

Vérifier sur jwt.io que le payload contient maintenant `"exp": <timestamp>`.

Test de validation

```
# Décoder le token et vérifier la présence de "exp"
TOKEN=$(curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' | jq -r '.token')

# Décoder le payload (base64)
echo $TOKEN | cut -d'.' -f2 | base64 -d 2>/dev/null | python3 -m json.tool
# Résultat attendu : champ "exp" présent avec valeur = now + 3600
```

Trace rapport > “Les tokens JWT sont désormais configurés avec une expiration de 1 heure (expiresIn: ‘1h’). Le champ ‘exp’ est présent et validé à chaque requête par le middleware d’authentification.”

5. Authentification ≠ Autorisation

Concept Authentification = vérifier l’identité (“qui es-tu ?”). **Autorisation** = vérifier les permissions (“as-tu le droit ?”). Ces deux étapes sont distinctes et doivent être implémentées séparément. Une app peut très bien authentifier correctement tous ses utilisateurs tout en donnant accès à des ressources non autorisées (c’est le cas de TaskFlow : le JWT est vérifié, mais les rôles ne sont pas contrôlés).

Risque concret Un utilisateur authentifié peut accéder aux données d’un autre utilisateur ou aux routes admin sans être administrateur.

Exemple de faille

```
// ⚠️ Faille : middleware vérifie seulement l'authentification, pas l'autorisation
router.get('/admin/users', authenticate, (req, res) => {
  // authenticate vérifie le JWT, mais N'IMPORTE QUEL utilisateur passe
  const users = db.prepare('SELECT * FROM users').all();
  res.json(users);
});
```

Action pratique Parcourir `routes/` et dresser un tableau :

ROUTE	AUTHENTIFICATION REQUISE	AUTORISATION REQUISE	IMPLÉMENTÉE ?
POST /login	☐	☐	☒
GET /tasks	☒	user.id === task.user_id	☐
GET /admin/users	☒	role === 'admin'	☐
DELETE /tasks/:id	☒	user.id === task.user_id	☐

Test de validation

```
# Token d'un utilisateur normal, ne doit pas accéder à /admin/users
USER_TOKEN=$(curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' | jq -r '.token')

curl http://localhost:3000/admin/users \
  -H "Authorization: Bearer ${USER_TOKEN}"
# Résultat attendu après correction : 403 Forbidden
```

Trace rapport > “Un audit des routes a identifié une confusion entre authentification et autorisation. Les routes sensibles n’effectuaient que la vérification du token JWT sans contrôle de rôle. Correction appliquée avec middleware dédié.”

6. RBAC (Rôles)

Concept Le **Role-Based Access Control** associe des permissions à des rôles (ex: `admin`, `user`) plutôt qu’à des individus. Le rôle est stocké dans le token JWT (claim `role`) et vérifié côté serveur à chaque requête sur les routes protégées. Le middleware de contrôle de rôle doit être séparé du middleware d’authentification pour être réutilisable.

Risque concret Sans RBAC, tout utilisateur authentifié peut accéder aux routes d'administration, exposant les données de tous les utilisateurs.

Exemple de faille

```
// ⚠️ Faille dans routes/admin.js
router.get('/admin/users', (req, res) => {
  const token = req.headers.authorization?.split(' ')[1];
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  if (decoded) { // ← vérifie seulement l'existence du token, pas le rôle
    const users = db.prepare('SELECT * FROM users').all();
    res.json(users);
  }
});
```

Action pratique

Créer `middleware/requireRole.js` :

```
// middleware/requireRole.js
function requireRole(role) {
  return (req, res, next) => {
    if (!req.user) {
      return res.status(401).json({ error: 'Non authentifié' });
    }
    if (req.user.role !== role) {
      return res.status(403).json({ error: 'Accès interdit, rôle insuffisant' });
    }
    next();
  };
}

module.exports = requireRole;
```

Appliquer dans `routes/admin.js` :

```
// AVANT (faille)
router.get('/admin/users', authenticate, (req, res) => { ... });

// APRÈS (corrigé)
const requireRole = require('../middleware/requireRole');
router.get('/admin/users', authenticate, requireRole('admin'), (req, res) => {
  const users = db.prepare('SELECT id, username, role, created_at FROM
users').all();
  res.json(users);
});
```

Test de validation


```
# Test 1 : utilisateur normal rejeté
USER_TOKEN="<token_alice>"
curl -w "\nHTTP: %{http_code}\n" http://localhost:3000/admin/users \
  -H "Authorization: Bearer ${USER_TOKEN}"
# Résultat attendu : HTTP: 403

# Test 2 : admin accepté
ADMIN_TOKEN="<token_admin>"
curl -w "\nHTTP: %{http_code}\n" http://localhost:3000/admin/users \
  -H "Authorization: Bearer ${ADMIN_TOKEN}"
# Résultat attendu : HTTP: 200
```

Trace rapport > “Un middleware `requireRole()` a été créé et appliqué sur `/admin/users`. Les utilisateurs non-admin reçoivent une réponse 403. Les tests confirment que seul le rôle ‘admin’ accède à la liste des utilisateurs.”

7. IDOR (Insecure Direct Object Reference)

Concept L’IDOR survient quand une ressource est accessible directement via un identifiant (ex: `/tasks/42`) sans vérifier que l’utilisateur courant est bien le propriétaire de cette ressource. L’attaquant itère sur les IDs (`/tasks/1`, `/tasks/2` ...) pour accéder aux données des autres. C’est la faille #1 du classement OWASP A01 (Broken Access Control).

Risque concret Alice (`userId=1`) peut lire, modifier et supprimer les tâches de Bob (`userId=2`) simplement en changeant l’ID dans l’URL.

Exemple de faille

```
// ⚠️ Faille dans routes/tasks.js
router.get('/tasks/:id', authenticate, (req, res) => {
  const task = db.prepare('SELECT * FROM tasks WHERE id = ?').get(req.params.id);
  if (!task) return res.status(404).json({ error: 'Tâche introuvable' });
  // ← AUCUNE vérification que task.user_id === req.user.id
  res.json(task);
});
```

Action pratique

```
// APRÈS (corrigé) dans routes/tasks.js
router.get('/tasks/:id', authenticate, (req, res) => {
  const task = db.prepare('SELECT * FROM tasks WHERE id = ?').get(req.params.id);
  if (!task) return res.status(404).json({ error: 'Tâche introuvable' });

  // ← Vérification propriétaire (IDOR fix)
  if (task.user_id !== req.user.id) {
    return res.status(403).json({ error: 'Accès interdit, ressource appartient à un
    autre utilisateur' });
  }

  res.json(task);
});
```

Appliquer le même pattern sur `PUT /tasks/:id` et `DELETE /tasks/:id`.

Test de validation

```
# Alice (userId=1) tente d'accéder à la tâche de Bob (id=2, user_id=2)
ALICE_TOKEN="<token_alice>"
curl -w "\nHTTP: %{http_code}\n" http://localhost:3000/tasks/2 \
  -H "Authorization: Bearer ${ALICE_TOKEN}"
# Résultat attendu après correction : HTTP: 403
# {"error":"Accès interdit, ressource appartient à un autre utilisateur"}
```

Trace rapport > “La route `GET /tasks/:id` était vulnérable à l’IDOR : tout utilisateur authentifié pouvait lire les tâches des autres utilisateurs. Correction appliquée : vérification `task.user_id === req.user.id` sur `GET`, `PUT` et `DELETE /tasks/:id`.”

8. Stockage du token côté client

Concept `localStorage` est accessible par tout JavaScript de la page, y compris un script injecté via XSS. Un attaquant qui injecte `document.cookie` ou `localStorage.getItem('token')` vole le token de l'utilisateur. La recommandation OWASP est de stocker le JWT dans un **cookie HttpOnly** : ce flag empêche tout accès JavaScript au cookie, neutralisant le vol de token par XSS.

Risque concret Une XSS quelconque dans l'app (même mineure) suffit à exfiltrer le token et usurper l'identité de l'utilisateur.

Exemple de faille

```
// ⚠️ Faille côté client (public/app.js)
fetch('/login', { method: 'POST', body: JSON.stringify(creds) })
  .then(r => r.json())
  .then(data => {
    localStorage.setItem('token', data.token); // ← accessible via XSS
  });
```

Action pratique

Modifier la route `/login` côté serveur (`routes/auth.js`) pour envoyer le token en cookie :

```
// APRÈS (corrigé) dans routes/auth.js
router.post('/login', (req, res) => {
  // ... vérification credentials ...
  const token = jwt.sign(
    { userId: user.id, role: user.role },
    process.env.JWT_SECRET,
    { algorithm: 'HS256', expiresIn: '1h' }
  );

  res.cookie('token', token, {
    httpOnly: true, // ← inaccessible au JS
    secure: process.env.NODE_ENV === 'production', // HTTPS uniquement en prod
    sameSite: 'strict', // protection CSRF
    maxAge: 3600000 // 1h en ms
  });

  res.json({ message: 'Connexion réussie' }); // NE PAS renvoyer le token dans le
  body
});
```

Modifier le middleware `authenticate.js` pour lire le cookie :

```
// middleware/authenticate.js (APRÈS)
const token = req.cookies?.token || req.headers.authorization?.split(' ')[1];
```

Test de validation

```
# Vérifier que la réponse /login contient Set-Cookie avec HttpOnly
curl -v -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' 2>&1 | grep -i "set-cookie"
# Résultat attendu : Set-Cookie: token=...; HttpOnly; SameSite=Strict

# Vérifier que le body NE contient PLUS le token
curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' | jq .
# Résultat attendu : {"message":"Connexion réussie"}, aucun champ "token"
```

Trace rapport > “Le token JWT était stocké dans localStorage, le rendant accessible via XSS. Correction : le token est désormais émis en cookie HttpOnly + SameSite=Strict. Le corps de la réponse /login ne contient plus le token.”

Atelier, Exploit IDOR complet puis correction

Scénario

L'étudiant possède deux comptes : `alice` (userId=1) et `bob` (userId=2). Bob a créé la tâche id=2. Alice va la lire sans autorisation.

Étape 1, Obtenir le token d'Alice

```
ALICE_TOKEN=$(curl -s -X POST http://localhost:3000/login \
  -H "Content-Type: application/json" \
  -d '{"username":"alice","password":"password123"}' | jq -r '.token')
echo "Token Alice: ${ALICE_TOKEN}"
```

Étape 2, Exploit IDOR (avant correction)

```
# Alice lit la tâche de Bob (id=2), succès avant correction
curl -s http://localhost:3000/tasks/2 \
  -H "Authorization: Bearer ${ALICE_TOKEN}" | jq .
# Résultat : la tâche de Bob s'affiche, FAILLE CONFIRMÉE
```

Étape 3, Correction dans le code

Appliquer la correction décrite en section 7 (vérification `task.user_id === req.user.id`).

Redémarrer l'application :

```
cd vuln-app && npm run dev
```

Étape 4, Re-test après correction

```
curl -w "\nHTTP: %{http_code}\n" http://localhost:3000/tasks/2 \
-H "Authorization: Bearer ${ALICE_TOKEN}"
# Résultat attendu : HTTP: 403
# {"error":"Accès interdit, ressource appartient à un autre utilisateur"}
```

TP, 65 minutes

Étape 1 : Audit du code auth/autorisation (10 min)

☐ Ouvrir `vuln-app/auth.js` et `vuln-app/middleware/authenticate.js` ☐ Lister les failles dans votre rapport : secret, expiration, stockage token, contrôle de rôle

Étape 2 : Exploit JWT faible, forger un token admin (10 min)

```
# Sur jwt.io > Debugger > coller un token existant
# Changer "role":"user" par "role":"admin" dans le payload
# Dans "Verify Signature", cocher "secret base64 encoded" = OFF
# Taper le secret "secret123" dans le champ "your-256-bit-secret"
# Le token modifié à droite est valide avec le secret faible
```

☐ Tester le token forgé sur `/admin/users` ☐ Documenter : “token forgé en X secondes avec secret faible”

Étape 3 : Exploit IDOR (5 min)

```
# Récupérer token d'alice
ALICE_TOKEN=$(curl -s -X POST http://localhost:3000/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"password123"}' | jq -r '.token')

# Accéder à la tâche de bob (id=2)
curl -s http://localhost:3000/tasks/2 \
-H "Authorization: Bearer ${ALICE_TOKEN}" | jq .
```

☐ Documenter le résultat dans le rapport

Étape 4 : Correction JWT secret + expiration (10 min)

☐ Générer le secret : `openssl rand -base64 32` ☐ Créer/modifier `vuln-app/.env` avec `JWT_SECRET=<secret_généré>` ☐ Modifier `auth.js` : remplacer `'secret123'` par `process.env.JWT_SECRET` ☐ Ajouter `expiresIn: '1h'` dans `jwt.sign()` ☐ Vérifier que `.env` est dans `.gitignore` ☐ Redémarrer l'app et vérifier que le token forgé est maintenant rejeté

Étape 5 : Middleware requireRole + correction /admin (15 min)

☐ Créer `vuln-app/middleware/requireRole.js` (code section 6) ☐ Appliquer `requireRole('admin')` sur `GET /admin/users` ☐ Tester avec un token user → 403 attendu ☐ Tester avec un token admin → 200 attendu

Étape 6 : Correction IDOR (10 min)

☐ Modifier `GET /tasks/:id` : ajouter vérification `task.user_id === req.user.id` ☐ Appliquer le même check sur `PUT /tasks/:id` et `DELETE /tasks/:id` ☐ Re-tester l'exploit IDOR de l'étape 3 → 403 attendu

Étape 7 : Tests de validation (5 min)

☐ Token forgé avec "secret123" → 401 ☐ Token sans expiration → toujours rejeté (car secret changé) ☐ IDOR Alice sur tâche Bob → 403 ☐ Utilisateur normal sur `/admin/users` → 403 ☐ Token en cookie HttpOnly (bonus)

Mini-quiz, 8 QCM

Q1. Un JWT est : - A) Chiffré et illisible sans clé privée - B) Encodé en Base64 et lisible sans secret - C) Stocké côté serveur dans une session - D) Toujours sécurisé si HTTPS est utilisé

Q2. Le secret JWT `"secret"` est problématique car : - A) Il est trop long - B) Il est prévisible et peut être bruteforcé - C) Il n'est pas compatible avec HS256 - D) Il doit être encodé en Base64 avant usage

Q3. La faille IDOR permet de : - A) Injecter du code SQL dans une requête - B) Accéder à des ressources appartenant à d'autres utilisateurs - C) Contourner HTTPS - D) Forger un token sans connaître le secret

Q4. Le middleware `requireRole('admin')` doit être placé : - A) Avant `authenticate` dans la chaîne de middlewares - B) Après `authenticate` dans la chaîne de middlewares - C) À la place de `authenticate` - D) Dans le contrôleur, pas en middleware

Q5. Stocker un JWT dans `localStorage` est risqué car : - A) `localStorage` est trop lent pour les tokens - B) `localStorage` est accessible via XSS et permet le vol du token - C) `localStorage` expire après 30 minutes - D) `localStorage` n'est pas supporté par tous les navigateurs

Q6. Un JWT sans `exp` : - A) Expire après 24h par défaut - B) Est invalide et rejeté par toutes les librairies - C) Est valide indéfiniment, faille si compromis - D) Déclenche une erreur à la signature

Q7. `sameSite: 'strict'` sur un cookie protège contre : - A) XSS - B) SQL injection - C) CSRF - D) Path traversal

Q8. Quelle est la différence entre authentification et autorisation ? - A) Aucune, ce sont des synonymes - B) L'authentification vérifie l'identité, l'autorisation vérifie les permissions - C) L'autorisation vérifie l'identité, l'authentification vérifie les permissions - D) L'authentification s'applique aux routes, l'autorisation aux données

Corrigé QCM

Q	RÉPONSE	EXPLICATION
1	B	Base64URL est un encodage, pas un chiffrement
2	B	Outils comme hashcat cassent les secrets courts en secondes
3	B	IDOR = accès non autorisé à un objet via son ID
4	B	<code>authenticate</code> doit peupler <code>req.user</code> avant que <code>requireRole</code> puisse le lire
5	B	XSS peut exécuter <code>localStorage.getItem('token')</code>
6	C	Pas de champ <code>exp</code> = token éternel
7	C	SameSite empêche l'envoi du cookie depuis un site tiers (CSRF)
8	B	Authentification = identité / Autorisation = permissions

Livrable

```
# Depuis la racine du dépôt, après toutes les corrections
git add vuln-app/
git commit -m "M3: fix JWT secret/expiration, RBAC middleware, IDOR checks,
HttpOnly cookie"
git tag M3-rbac-securise
git push origin main --tags
```

Ressources

- OWASP A01:2021, Broken Access Control : https://owasp.org/Top10/A01_2021-Broken_Access_Control/
- OWASP A07:2021, Identification and Authentication Failures : https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- JWT Best Practices (RFC 8725) : <https://datatracker.ietf.org/doc/html/rfc8725>
- OWASP JWT Security Cheat Sheet : https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html